

# ATDM Notes

Notes over the articles ATDM, Part1, Part2, and ROM.

I attempted to clarify the implicit assumptions and produce concepts that best unify across articles.

## Formal Language

The language is inherited from Predicate Logic. Two new (semantically meaningful) symbols are introduced, which are  $\subseteq$  (inclusion) and  $\otimes$  (relation).

### Axiom of Objects

In ATDM, this axiom states that everything in the universe is an object. It does not introduce new mathematics, i.e. the resulting system is isomorphic to a system without this axiom. Philosophically though, it can act as a means of connecting reality and mathematics. This axiom is not used in Part1/Part2.

ATDM claims to be built from set theory, but it shares more similarity with mereology. If so, the entire axiomatic system of mereology must be explicitly assumed in later publications.

## Axiom of Extensionality

Two objects are equal if they include the same objects. Equality merely allows substitution (standard requirement in most axiomatic systems).

$$\forall A, B : (\forall X : X \subseteq A \leftrightarrow X \subseteq B) \rightarrow A = B$$

In ATDM, identity/equality is simply a definition, which is not sufficient for substitution.

Concept is absent from Part1/Part2.

## Axiom of Bottom

There exists a null object.

$$\exists B, \forall X : B \subseteq X$$

By extensionality, it is unique. We denote it with  $\Phi$ .

The null object is frequently used in Part1, but never ontologically legitimized.

It is not used in ATDM.

## Axiom of Union

We can take the union of two objects.

$$\forall A, B, \exists C, \forall X : X \subseteq C \leftrightarrow (X \subseteq A \vee X \subseteq B)$$

In ATDM, union is simply a definition. As an analogy, we can keep talking about unicorns "if they existed", but it doesn't legitimize their ontology.

In ATDM, intersection is only used to define a "range" operator. Professor Zeng mentioned that it has been dropped, so we are omitting everything involving range in this article.

Absent from Part1/Part2.

## Axioms of Relation

For each pair of objects, there exists a unique object representing their specific relation. Note that it is not necessarily commutative.

$$\forall A, B, \exists!(A \otimes B)$$

Law of relation over union:

$$A \otimes (B \cup C) = (A \otimes B) \cup (A \otimes C)$$

$$(A \cup B) \otimes C = (A \otimes C) \cup (B \otimes C)$$

In ATDM, the corollary 2.1 states that objects must include other objects, claiming that inclusion is a relation. However, there is no way to prove this corollary, even with this claim.

Additionally, the laws are defined in a later location in ATDM, but they are not listed as axioms. Since, in practice, the relation operator is always used in conjunction with these laws, I've moved them here, stating their axiomatic nature for practicality.

Relations are somewhat absent from Part1/Part2, because performance is defined loosely, and later defined as Cartesian products.

## Theorem of Structure

The structure of an object exists, which is the union of the object with its self-relation.

$$\oplus A = A \cup (A \otimes A)$$

Proof:

Relation and union exist, hence the formula produces a valid set.

Formalization from ATDM. Absent from Part1/Part2.

## Theorem of Object Decomposition

The structure of object union is the union of sub-objects structure with relations between sub-objects.

Proof:

$$\begin{aligned}
 & \oplus \bigcup_{\alpha} O_{\alpha} \\
 &= \left( \bigcup_{\alpha} O_{\alpha} \right) \cup \left( \bigcup_{\alpha} O_{\alpha} \otimes \bigcup_{\alpha} O_{\alpha} \right) \\
 &= \left( \bigcup_{\alpha} O_{\alpha} \right) \cup \left( \bigcup_{\alpha, \beta} O_{\alpha} \otimes O_{\beta} \right) \\
 &= \left( \bigcup_{\alpha} \oplus O_{\alpha} \right) \cup \left( \bigcup_{\alpha \neq \beta} O_{\alpha} \otimes O_{\beta} \right)
 \end{aligned}$$

Compared to ATDM's corollary 2.1, which assumes that there exists sub-objects  $O = \bigcup_{\alpha} O_{\alpha}$ , this theorem begins with an object that is already expressed as a union of sub-objects. The union axiomatically exists, so this version of the theorem is solid.

ROM is entirely based around this theorem.

Part1/Part2 also use it extensively, although never proved.

In Part2, "type" is a confusing notion. It denotes both abstraction (type) and inclusion (sub-object). Clearly, a leaf is not a *type* of tree, even though it's *part* of a tree.

## Axiom of World

There exists an object called "world" equal to the union of two objects, one of which is finite. The finite object  $M$  is called "human thought". The other,  $N$  is called "nature".

$$\exists W = N \cup M$$

If we decompose its structure  $\oplus W = (\oplus N) \cup (\oplus M) \cup (N \otimes M) \cup (M \otimes N)$ .

$\oplus N = N \cup (N \otimes N)$  is called the "natural system".  $L \subseteq N \otimes N$  is called "natural law".

$\oplus M = M \cup (M \otimes M)$  is called the "human rational system".  $K \subseteq M \otimes M$  is called "knowledge".

One of the subsets  $f^R \subseteq N \otimes M$  is called recognition.

One of the subsets  $f^A \subseteq M \otimes N$  is called human action.

When applying  $f^R(x) = y \in M$  where  $x \in N$ , we use the abbreviation  $x \approx y$  and we say that they are correlated.

Again, in ATDM, the world is simply a definition, so here I ontologically legitimize it. Also, I've merged many of the subsequent definitions into this axiom for concision.

In ATDM, there's an additional axiom called the Axiom of Bounded Rationality, which states that the amount of sub-objects in the human mind is finite. I've merged it with the Axiom of World for convenience.

Also, the concept of finiteness is lacking formalization in ATDM. It is also missing in the present article. It will be worked on later.

Also, the concept of function is murky here. Either functions must be axiomatized separately, or they must be defined in terms of mereological sets. The latter is currently beyond my abilities. The former might better adhere to synthetic mathematics <https://ncatlab.org/nlab/show/synthetic+mathematics>.

Also, in ATDM, there is an Axiom of Recognition. However, if we can formalize functions, then the axiom becomes a theorem. The idea of homomorphism is interesting here.

Also, there's the notion of causal relation in ATDM. When objects are designated as causes and effects, the axiom states the existence of a relation where effect is unique per cause. Again, it overloads the relation operator, seemingly contradicting the Axiom of Relation.

Finally, the notion of "correlation class" is used in Part1. I've tried to clarify it here. Its symmetry property is never used, so I've omitted it.

Concept is nascent in Part1/Part2 through concept of knowledge, but not formalized yet.

## Theorem of Engineering System

Let an object  $S \subseteq N$  be composed of finitely many sub-objects. (Note: probably needs an axiom here.)  $S$  is called the product. Let the environment be the complement  $E = N - S$ . In this context, the structure of nature is called the engineering system. It's composed of the product structure, the environment structure, and the relationship between product and environment.

Proof:

$$\begin{aligned}\oplus N &= \oplus(S \cup E) \\ &= (S \cup E) \cup ((S \cup E) \otimes (S \cup E)) \\ &= (S \cup E) \cup ((S \otimes S) \cup (S \otimes E) \cup (E \otimes S) \cup (E \otimes E)) \\ &= (\oplus S) \cup (\oplus E) \cup (S \otimes E) \cup (E \otimes S)\end{aligned}$$

Using the Theorem of Object Decomposition, since sub-objects are finite, we can represent a product using a tree. A node is a triple  $(i, j, k)$  where  $i$  is the node's level,  $j$  is the node's index relative to every cousin (including siblings), and  $k$  is the parent's  $j$ . We call  $i$  the "complexity level".

In Part2, these sub-objects are ordered pairs (name, value), which would require an additional axiom. There is also a distinction between components and connectors.

## Theorem of Performance Decomposition

Given an engineering system, a subset  $A \subseteq E \otimes S$  is called environment action, and a subset  $R \subseteq S \otimes E$  is called product response. A subset  $P \subseteq A \otimes R$  is called product performance. Performance of  $S$  can be analyzed by looking at the cross-performance between every pair of sub-object  $S_\alpha, S_\beta$ .

Proof:

$$\begin{aligned} P &\subseteq (S \otimes E) \otimes (E \otimes S) \\ &= \bigcup_{\alpha, \beta} (S_\alpha \otimes E) \otimes (E \otimes S_\beta) \end{aligned}$$

A clearer formula is

$$\bigcup_{\alpha} \left( P_\alpha \cup \left( \bigcup_{\alpha \neq \beta} (S_\alpha \otimes E) \otimes (E \otimes S_\beta) \right) \right)$$

In other words, to analyze a product's performance, you can analyze each  $\alpha$ -th sub-object's performance and their cross-performance with other sub-objects.

In earlier articles like Part1,  $P$  is assumed to follow from  $S$ , but the idea of using the relation operator to explicitly describe  $P$  in terms of  $S$  was not discovered yet.

In Part2, each action or response is defined as a specific subset of a product's behavioral property. In other words, an action/response cause (possibly many) behavior from the product. ATDM only makes the idea more precise, so the ATDM version is retained here.

In ATDM, there is a theorem stating the finiteness of performances. However, it is lacking the application of  $f^R$  onto actual product performance, in order to work inside human thoughts with the homomorphic finite performance. Because only the human mind is finite. Haven't formalized it myself yet, but it's easy.

The core issue in Part1, which has propagated into ATDM, is that product performance can be decomposed into the union of sub-object performances, which ignores the entire cross-performance, which is the  $S_\alpha \otimes E$  and  $E \otimes S_\beta$  portion. Part1 does attempt to remedy this problem. See last axiom.

An idea to directly fix this problem is to introduce an axiom that looks like:

$$\forall (A \otimes B), \forall C : (A \otimes C) \otimes (C \otimes B) \subseteq A \otimes B$$

i.e. intermediate relations are a subset of the original relation, but it's still unclear.

# Definition of Constraint

A collection of  $\beta$  (possibly uncountable) logical statements involving  $X = S \cup P_S$

$$R_\beta(X, [X]_\beta)$$

are called design specifications/requirements. In this context, we abbreviate their conjunction by

$$\lambda(X, [X]) = \bigwedge_{\beta} R(X, [X]_\beta)$$

In other words, each  $[X]_\beta$  constraints  $X$  in one aspect and we wish to satisfy every aspect's constraint. Each  $X_{\beta_i}$  is basically a valid candidate, where the final one is chosen by elimination.

In Part1, constraints/requirements/specifications (interchangeable) are defined incoherently. In one view,  $\lambda$  is seen as a predicate of  $X$  and  $[X]$ , which should yield a logical value (i.e. True or False), i.e. if  $X$  satisfies  $[X]$ . However, in another view, they produce another object  $R^d = \lambda(X, [X])$  which is called "design specifications". If so, then  $R^d$  is a boolean value, but it's not used in such a way throughout the article. In other words,  $\lambda$  is both an evaluation operator and a design requirement generator, which is ambiguous.

In this article, I decided to stick with the predicate view, since its only used as such, reserving  $\lambda^*$  for the design requirement generator. Hence why I used the conjunction operation and the logical statement approach.

Also, in Part1, the  $\lambda$  operator can yield three values: 1, 0, -1. These are interpreted as True, False, and Undetermined. Since Undetermined behaves exactly like False in this theory, I've discarded Undetermined. However, on a conceptual level, Undetermined clarifies the procedure to take when a designer is not sure about the product, so it does have its merit.

ATDM also talks about constraints, but they rely on the range operator, so we will ignore ATDM.

In Part2, there is a separation between structural and behavioral property/requirement, which is probably the ancestor of the current ATDM approach of product and performance.

## Axiom of the Design Governing Equation

There exists a function  $\lambda^*(X_{t=i})$  which yields the requirements that a human decides to impose on  $X = S \cup P_S$ . Note that  $\lambda^*$  is different from  $\lambda$ . There exists a function  $S(\lambda)$  yielding a product  $S_{t=i+1}$  satisfying requirements  $\lambda^*$ . A product  $S_{t=t_f}$  is considered finished when it satisfies the Design Governing Equation:

$$S_{t=t_f} = S(\lambda^*(P(S_{t=t_f})))$$

where we use the shorthand  $P(S) = S \cup P_S$ . In other words, a finished product is a stationary point of the equation.

This equation can only be solved within the human rational system. Hence, we have a homomorphic equation, where each object is the application of  $f^R$  onto its natural counterpart.

$$K_{t=t_f}^S = K^S(\lambda^*(K^P(K_{t=t_f}^S)))$$

Note that  $K^S$  is overloaded with both product knowledge and product synthesis. In design context, every knowledge  $K$  must be acquired through  $f^R$ .

Note: in Part1, the notation is slightly different from mine, but the concepts remain the same.

In Part1, since it's not rigorous, this is not formalized as an axiom. Clearly, it is an axiom, so I made it an axiom.

In Part1, there is no distinction between nature and human thought. However, its usage of the concept of knowledge seems to be the ancestor. For example, it talks about acquiring knowledge, limited primitive products, etc, which are related to the formalized version in ATDM. Here, I've formalized it in the spirit of ATDM.

Notation-wise, in Part1 uses  $\lambda^*$  in a nonsensical way. It defines  $\lambda^*(X)$  as the value of  $\lambda(X, [X])$  when  $\lambda(X, [X]) = 1$ , which means  $\lambda^*$  simply always equals 1 (or True). This reflects the ambiguous definition of  $\lambda$  previously mentioned.

About performance knowledge, recall correlation classes in Part 1. The first mention of correlation classes actually occurs here. Since an object is in nature, its performance/behavior is a subset of natural law, and our knowledge of these behaviors are correlated with their real behavior.

The concept of design governing equation itself is absent in ATDM.

**Warning! This section summarized from Part1 relies on the incorrect version of performance decomposition in ATDM/Part1:**

Since human thoughts are finite, we have a set of primitive sub-objects  $K_\alpha^S$ . By performance decomposition, each must satisfy their own primitive equation

$$K_\alpha^S = K_\alpha^S(\lambda_\alpha^*(K_\alpha^P(K_\alpha^S)))$$

By the way,  $K_\alpha^P$  contains  $P_\alpha$ , which is called primitive performance of  $S_\alpha$ .

Note: In Part1,  $K_\alpha^P \subseteq S_\alpha \times P_\alpha$ , which is probably incorrect. We want the result of  $K_\alpha^P$  to contain both the product and its performance, but the Cartesian product notation means that it only yields the performance.

## Amendment attempt from Part1

Here, we talk about the unwanted performance of sub-objects. We can decompose design specifications into primitive specifications for each  $S_\alpha$ . We dispense with the  $K$  notation; assume that everything happens inside human thoughts.

$$R = \bigcup_{\alpha} R_{\alpha}$$

We use  $A \updownarrow B$  to denote "conflict" between sub-objects augmented with their performance (i.e.  $X = S \cup P$ ), which consists of a set of new requirements which would remove the current unwanted performances generated by them.

We have the following entire design process algorithm:

1.  $R_{t=0} = R$
2.  $S_{t=0} = \Phi$
3. Iterate over  $t$  until  $R_t = \Phi$ :
  1. Find  $S_\alpha$  satisfying the primitive design equation for some  $\alpha$  (order not specified).
  2. Combine the primitive product with the current partial product  $S_t = \xi(S_{t-1}, S_\alpha)$ .
  3. Find the conflict  $R' = P(S_t) \updownarrow P(S_\alpha)$ .
  4. Append the new requirements  $R_t = R_{t-1} \cup R'$ .
4.  $S = S_t$

This amendment attempt is conceptually good, but it's not formalized.

Note: In Part1, they use the decomposition operator  $\tau$ , but it's never used in the design process algorithm, so I omit it here.

The ROM article doesn't seem to introduce anything new.